Intro



안녕하세요, 김종재입니다.

1995. 07. 03

gghs703@naver.com / 01053901348

GitHub Link

https://github.com/KimJongjae1

제조 현장의 업무 흐름을 이해하고, 데이터를 기반으로 문제를 해결하는 개발자를 지향합니다.

웹 및 PDA 기반 MES 업무 화면을 개발하며 바코드 스캔, LOT 검증, 재고 이동, 입고, 실사 처리와 같은 현장 중심 기능을 구현했습니다. 단순히 화면을 만드는 것에 그치지 않고, 사용자가 어떤 순서로 작업하고 어떤 지점에서 오류가 발생할 수 있는지 고려하며 검증 로직, 중복 처리, 사용자 피드백을 설계했습니다.

문제가 발생했을 때는 테스트 결과와 로그 데이터를 바탕으로 원인을 추적하고, 개선 방향을 확인하는 방식을 중요하게 생각합니다. 앞으로도 제조 도메인의 데이터를 정확하게 다루고, 현장에서 신뢰할 수 있는 시스템을 만드는 개발자로 성장하겠습니다.

주요 기술

C#	조건 분기, 데이터 검증, 예외 처리 흐름을 고려해 안정적인 서비스 로직을 작성할 수 있습니다. MES/업무 시스템 환경에서 데이터 처리 흐름과 화면 기능을 연결하는 로직 구현 경험이 있습니다.
React	컴포넌트 기반으로 화면을 구성하고, useState, useEffect, useRef 등을 활용해 상태와 이벤트 흐름을 관리할 수 있습니다. 사용자의 업무 흐름을 고려해 조회, 스캔/입력, 검증, 저장으로 이어지는 화면 로직을 구현할 수 있습니다.
MS SQL Server	업무 데이터 흐름을 이해하고, 조회/검증/저장/삭제 로직에서 필요한 데이터를 SQL 기반으로 작성할 수 있습니다. 로그 및 Profiler를 근거로 오류 원인을 추적하고, 테이블 조회, 데이터 확인, 쿼리 작성 및 프로시저 호출 결과를 분석할 수 있습니다.
Java	알고리즘(백준 플 4), 자료구조 기반 구현이 탄탄하고, 서비스 로직을 안정적으로 작성 가능한 수준
SpringBoot	REST API 설계, Spring Security + JWT 인증, JPA 기반 CRUD/검색, 트랜잭션 처리
MySQL	ERD/스키마 설계, 인덱스 고려 쿼리 작성, 실행계획 기반 성능 점검 경험

교육

삼성 청년 소프트웨어 아카데미(SSAFY)

2025. 01. 01 - 2025.12.18

경력

에이시에스(MES 기술부)

2026. 02. 02 ~ (재직중)

셀트리온제약 (생산관리)

2022. 06. 27 - 2024. 09. 26 (2년 3개월)

학력

건국대학교 서울캠

2015- 2021 (융합생명공학과)

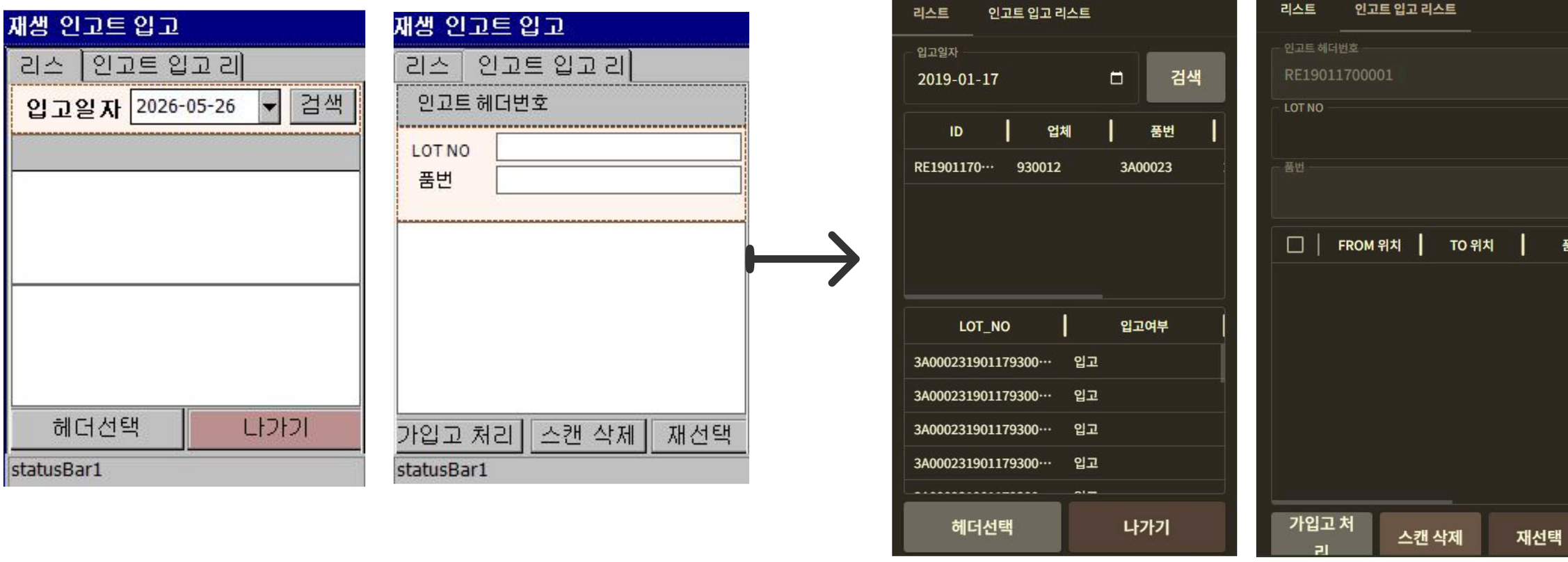
자격증

정보처리기사

취득일:2025.12

핸즈코퍼레이션

Project - 1



React 기반 Android PDA 마이그레이션 프로젝트 (구형 PDA C# 코드)

요약

기존 C# 코드로 되어있는 구형 PDA 코드를 React Native 환경에서 동작하는 React 기반 Android PDA 업무 화면을 개발했습니다. 작업자가 PDA 장비로 LOT 바코드를 스캔하여 자재/완제품의 입고, 공장 이동, 재고 실사 업무를 처리할 수 있으며 스캔 데이터를 수신하고, 서버 프로시저 API를 통해 LOT 검증, 중복 체크, 데이터 저장, 처리 결과 알림까지 이어지는 현장형 업무 플로우를 구현했습니다.

작업 기간 구성원-5명

2026. 04. 20 - 2026. 05. 15 (4주)

역할

재생 인고트 입고, 완제품 입고, 완제품 공장 이동 입고, 재고 실사 등 물류, 재고 처리 화면을 개발

React Native WebView 환경에서 웹 화면 구동

단위 및 통합 테스트 진행 (웹 및 PDA)

프로시져 검토 후 수정 및 예외사항 추가

프로젝트 회고

- **Android PDA에 대한 이해**
- Native앱은 React Native WebView 안에 React Web 화면을 띄우고 Zebra PDA의 DataWedge 스캔 데이터를 WebView로 전달
- API는 프로시저명과 파라미터를 전달하면, API 서버가 공장별 DB 연결 정보를 기준으로 Stored Procedure를 실행하고 결과를 JSON 형태로 반환
- API 설계도 중요하지만 예외사항 처리의 중요성을 배움
- 테스트의 중요성 / 테스트 하면서 생각이상으로 많은 버그가 발견됨.

핸즈코퍼레이션

Project - 1

원자재관리	버진인고트 입고	INGOT	frmStock_INGOT_INPUT	2	표, 입력, 저장
원자재관리	재생인고트 입고	REINGOT	frmStock_REGENERATIVE_INGOT_INPUT	2	표, 입력, 저장
원자재관리	인고트이동처리	INGOTMOVE	frmStock_MATERIAL_MOVE	2	표, 입력, 저장
공장간 이동	재공 출고	MOVE_OUT	frmProduction_Plant_Move("0")	2	표, 입력, 저장
공장간 이동	재공 입고	MOVE_IN	frmStock_PLANT_MOVE_INPUT("0")	2	표, 입력, 저장
공장간 이동	완제품 출고	MOVE_OUT_FINISH	frmProduction_Plant_Move("1")	2	표, 입력, 저장
공장간 이동	완제품 입고	MOVE_IN_FINISH	frmStock_PLANT_MOVE_INPUT("1")	2	표, 입력, 저장
생산관리	대기품 투입	READYITEM	frmProduction_WATING_ITEM_INPUT2	2	표, 입력, 저장
생산관리	대기품 투입 취소	READYITEM_CANCLE	frmProduction_WATING_ITEM_INPUT2_CANCLE	2	표, 입력, 저장
생산관리	대기품 투입조회	READYITEM_SEARCH	frmProduction_WATING_ITEM_INPUT_SEARCH	2	조회, 검색
관리부	완제품입고	FINISHED_IN	frmProduct_FINISHED_MOVE	2	표, 입력, 저장
관리부	완제품출고(물류창고)	FINISHED_OUT	frmProduct_FINISHED_MOVE2	2	표, 입력, 저장
관리부	출하처리	SHIPMENT	frmShipment_Shipment_Process	3	바코드, 다중 프로시저, 복잡
관리부	구역이동	MOVE_CELL	frmProduct_WAREHOUSE_MOVE	2	표, 입력, 저장
외주관리	외주 출고	OS_OUT	frmOS_TRANS_OUT	2	표, 입력, 저장
외주관리	외주 입고	OS_IN	frmOS_TRANS_IN	2	표, 입력, 저장
금형관리	금형이동처리	MOLDMOVE	frmMolding_Move	2	표, 입력, 저장
금형관리	금형폐기처리	MOLDTERMINATE	frmMolding_Terminating	2	표, 입력, 저장
자재관리	자재불출	STOCKOUT	frmStock_STOCK_USAGE	2	표, 입력, 저장
자재관리	재고실사	STOCKINSPECTION	frmStock_Inspection	2	표, 입력, 저장

테스트

각 화면별로 조회, 바코드 스캔, 데이터 추가, 중복 검증, 셀 클릭, 선택 삭제, 저장 처리, 나가기 기능을 기준으로 테스트 케이스를 작성하고 검증 진행했습니다.

PDA 화면은 단순 버튼 동작보다 실제 현장 사용 흐름이 중요하다고 판단하여, LOT 바코드 스캔 후 API 검증, 그리드 반영, 체크박스 선택 상태, PLT 수량/총 수량 계산, 저장 후 화면 초기화까지 이어지는 전체 흐름을 중심으로 테스트 진행하였습니다.

발견된 문제들

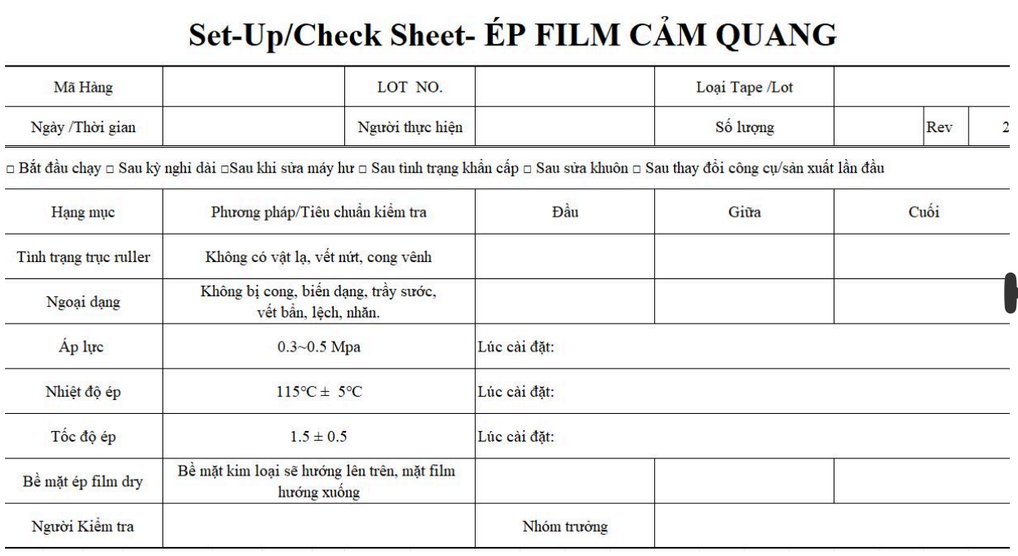
- 스캔 성공 후 신규 row가 자동 체크되지 않는 문제
- 중복 스캔 또는 검증 실패 후 LOT NO, 품번 입력값이 초기화되지 않는 문제
- 전체 선택 후 삭제 시 선택 모델 처리 오류로 삭제가 정상 동작하지 않는 문제
- 저장 처리 후 그리드와 입력 필드가 초기화되지 않는 문제
- 나가기 버튼 로직이 누락된 화면 확인
- 페이지 진입 시 날짜 기본값이 설정되지 않는 문제
- 창고/공장 드롭다운 초기값이 로그인 공장 기준과 맞지 않는 문제
- 비동기 초기화 함수가 완료되기 전에 다음 로직이 실행되는 문제
- DataGrid 컬럼명, From/To 표시값, 수량 합계 계산이 기존 WinForm 기준과 다른 문제
- 위치 정보가 없거나 처리 불가능한 LOT에 대한 사용자 알림이 부족한 문제

TT-13	출하처리	PDA	리스트	66	조회	출하일자를 선택한 뒤 [검색] 버튼을 누른다. 결과: 출하지시 목록 그리드에 데이터가 표시되고, 첫 번째 출하지시의 상세 정보가 하단 그리드에 표시된다. 조회 조건에 맞는 데이터가 없으면 "해당 조건에 맞는 출하지시 정보가 없습니다." 팝업이 표시된다. ● 테스트 날짜: 2019-03-25	o		1. 초기에 조회시 날짜가 비어있는 상태로 조회되는 문제 (해결함) 2. 처음에 지시선택을 하고 그 후, 다시 날짜를 조회 후 해당 날짜에 출하지시 목록이 없음에도 불구하고, 출하처리, 스캔삭제, 나가기 버튼이 활성화되는 문제가 있습니다. 검색 버튼 눌렀을 때 모든 Datagrid를 초기화를 시켜야합니다.(해결)
				67	셀 클릭	출하지시 헤더 그리드의 셀 또는 행을 클릭한다 선택한 출하지시의 상세 품목 정보가 하단 그리드에 조회된다.	o		handleRowClick 성공 여부를 true/false로 반환해서, 실패 시 selectOnClick이 실행되지 않게 해야 함 즉 셀클릭이 실패해서 detail이 잘못 업로드 되어도 있는걸로 인식함. ()---해결
				68	[지시선택]	출하지시 헤더를 선택한 뒤 [지시 선택] 버튼을 누른다. 출하지시 탭으로 이동하고 선택한 출하지시번호가 표시된다. 기존 임시 스캔 내역이 있으면 스캔 리스트에 표시되고 체크 상태가 된다. 출하지시 목록 또는 상세 품목 정보가 없는 상태에서 [지시 선택] 버튼을 누른다. 결과: 출하 처리할 데이터가 없다는 팝업이 표시된다. ● 테스트 입력값: 출하지시 목록: 819032500123 ● 테스트 입력값: 출하지시 목록: 819032500124	o		
				69	[나가기]	홈 화면으로 진입한다.	o		
			출하지시	70	추가(스캔)	출하지시를 선택한 뒤 [지시 선택] 버튼을 누른다. 출하지시 화면으로 이동하고 LOT NO를 입력하거나 바코드를 스캔한다. 검증 성공 시 스캔 리스트 그리드에 LOT 정보가 추가되고 체크 상태가 된다. PLT 수량은 리스트 건수로 갱신되고, 잔여/스캔 수량이 재계산된다. (탭 0에서 스캔했을때) ● 테스트 입력값: 작업지시서 바코드: 819032500123 ● 테스트 입력값: 작업지시서 바코드: 819032500124 (탭 1에서 스캔했을때) ● 테스트 입력값: LOT번호 바코드: GM0055301812112002 ● 테스트 입력값: LOT번호 바코드: GM0034201811241001 이미 리스트에 추가된 LOT를 다시 스캔한다. 결과: 중복 검증 메시지가 팝업으로 표시되고 중복 추가되지 않는다. LOT NO가 비어 있거나 처리할 수 없는 LOT를 입력한다. 결과: 검증 실패 메시지가 팝업으로 표시되고 LOT 입력값이 초기화된다.	o		1. 지시 선택 후, 출하 지시 탭에서 수동으로 LOT 번호를 입력시 "출하지시 페이지에서 작업해주세요" 라는 오류가 뜨고 있습니다. (useEffect 추가로 수정 완료) 2. 출하지시번호가 선택되지 않은 상태에서 탭 1에서 바코드를 스캔할 경우, 정상적인 동작은 출하지시번호가 없으므로 이후 로직을 수행하지 않아야 한다. 그러나 실제로는 출하지시번호가 없는 상태에서도 스캔 로직이 실행되는 문제가 발생하였다. 이에 따라 스캔 로직 내에서 출하지시번호 텍스트 값이 비어있는지 검증하도록 수정하였으며, 값이 없을 경우 에러 메시지를 출력한 뒤 return 처리하여 이후 로직이 실행되지 않도록 개선하였다. (수정 완료)
				71	[내역삭제]	"정말 삭제 하시겠습니까?" 확인 팝업이 표시된다. Yes 선택 시 체크된 항목만 삭제되고 PLT 수량/총 수량이 재계산된다. No 선택 시 현재 리스트를 유지한다. 리스트가 비어있는 상태거나 선택하지 않는 상황에서 [내역 삭제] 버튼을 누른다. 삭제할 데이터가 없으므로 LOT NO / 품번 입력값만 초기화되고 리스트는 그대로 유지된다.	o		작업지시서 하나를 선택한 후 페이지 1로 이동하여 해당 지시서의 LOT 번호를 입력한 뒤, 다시 페이지 0으로 이동해 다른 작업지시서를 선택하고 페이지 1로 진입한 상태에서 기존 입력된 LOT 번호를 삭제할 경우, 실제 삭제 프로시저는 호출되지 않지만 UI 상에서는 삭제된 것처럼 보이는 문제가 발생하였다. 실제 로직에서는 if 문으로 조건 처리가 되어 있었으나, deleteApi() 함수 내부의 selectedDevOrder 처리 로직에서 return만 수행하고 있어 이후 처리 로직이 실행되지 않는 문제가 있었다. 이로 인해 실제 데이터는 유지된 상태에서 UI만 삭제된 것처럼 표시되었다. 해당 문제는 return 1;로 수정하여 이후 로직이 정상적으로 수행되도록 개선하였다. (해결함)
				72	셀 클릭	스캔 리스트 헤더 체크박스를 클릭한다. 리스트 항목의 체크 상태가 전체 선택 또는 전체 해제로 토글된다.<	o		
				73	[출하처리]	정상 스캔된 LOT가 있는 상태에서 [출하처리] 버튼을 누른다. "정말 출하처리를 하시겠습니까?" 확인 팝업이 표시된다. Yes 선택 시 출하처리 저장 API가 호출되고, 성공하면 리스트가 초기화되며 화면 상태가 초기화된다. ● 테스트 입력값: 작업지시서: 819032500123 / LOT 번호: GM0055301812112002 ● 테스트 입력값: 작업지시서: 819032500124 / LOT 번호: GM0055301812112001 스캔 리스트가 비어 있는 상태에서 [출하처리] 버튼을 누른다. 결과: 출하 처리할 데이터가 없다는 팝업이 표시되고 저장되지 않는다.	o		initOnClick()을 await하지 않음 출하 성공 후 초기화, 선점 해제, 목록 재조회가 끝나기 전에 handleShipment의 finally 수행될수있음 (해결)
				74	[나가기]	초기 로딩 상태랑 똑같이 상태가 복구되어야한다.	o		유효상 하는 중 initOnClick 안의 initFetchDatas(1)을 await하지 않음. 나가기 후 날짜 비동기 문제와도 연결됨. (해결함)

테스트 예시

썬텍(베트남 공장 신규 mes 구축)

Project - 2

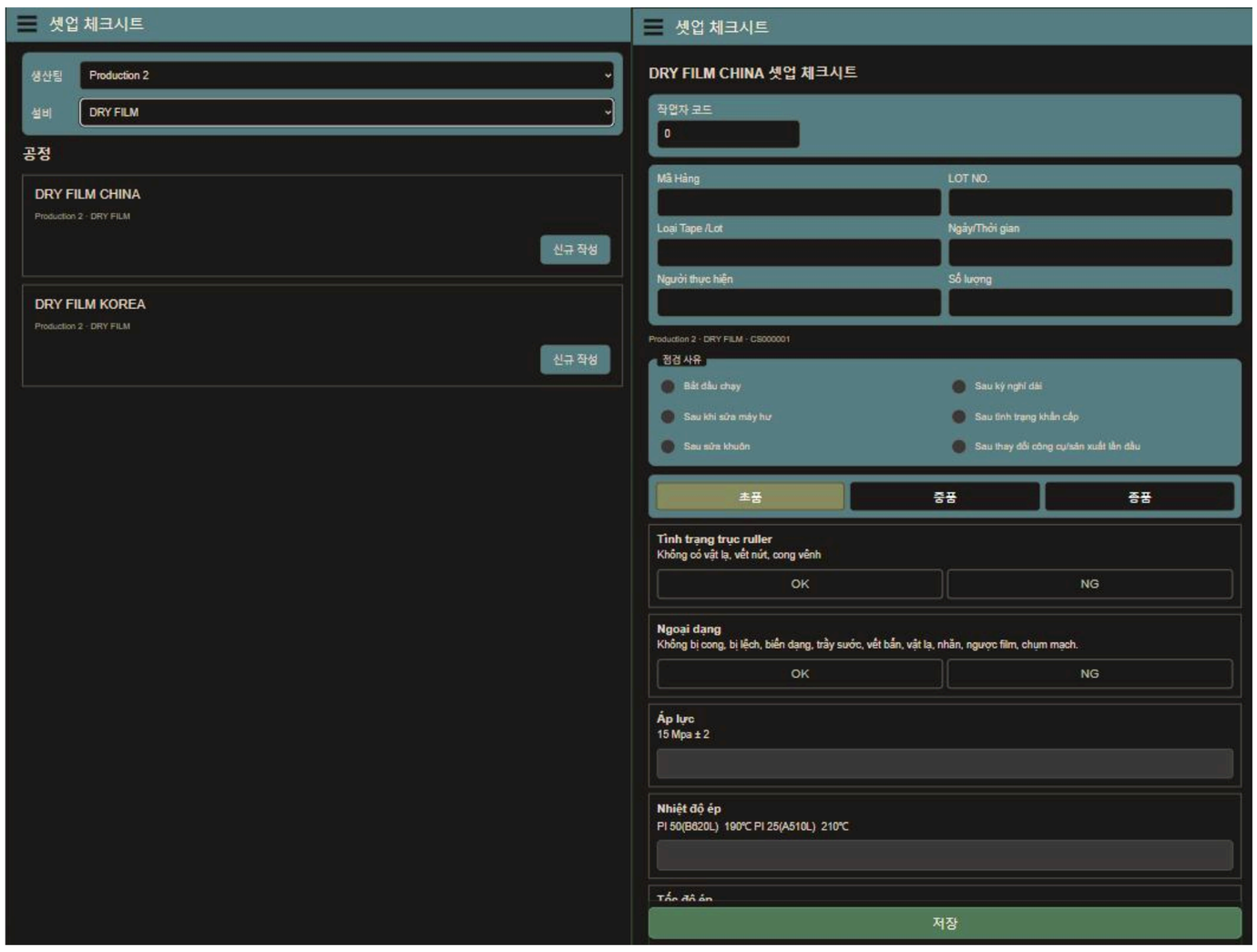


Set-Up/Check Sheet- ÉP FILM CẢM QUANG

Mã Hàng		LOT NO.		Loại Tape /Lot			
Ngày /Thời gian		Người thực hiện		Số lượng		Rev	2

□ Bắt đầu chạy □ Sau kỳ nghỉ dài □Sau khi sửa máy hư □ Sau tình trạng khẩn cấp □ Sau sửa khuôn □ Sau thay đổi công cụ/sản xuất lần đầu

Hạng mục	Phương pháp/Tiêu chuẩn kiểm tra	Đầu	Giữa	Cuối
Tình trạng trục ruller	Không có vật lạ, vết nứt, cong vênh			
Ngoại dạng	Không bị cong, biến dạng, trầy sước, vết bẩn, lệch, nhăn.			
Áp lực	0.3~0.5 Mpa	Lúc cài đặt:		
Nhiệt độ ép	115°C ± 5°C	Lúc cài đặt:		
Tốc độ ép	1.5 ± 0.5	Lúc cài đặt:		
Bề mặt ép film dry	Bề mặt kim loại sẽ hướng lên trên, mặt film hướng xuống			
Người Kiểm tra		Nhóm trưởng		



요약

수기로 작성하던 제조 현장의 셋업 체크리스트를 MES 업무 화면으로 전산화했습니다.
작업자가 생산팀과 설비에 맞는 체크시트를 선택하고, 작업 기본정보와 점검 사유, 초품/중품/종품 검사 결과를 입력 및 저장할 수 있도록 구현했습니다.
또한, 초품/중품/종품 검사에 맞게 공정별 검사 항목이 변화되게 하고 항목의 특성에 따라 OK/NG 선택과 측정값 입력을 구분했습니다.

역할

생산팀,설비별 셋업 체크시트 선택 및 신규 작성 화면 개발
작업 기본정보와 점검 사유 입력 기능 구현
초품,중품,종품 전환에 따른 검사 항목 표시 처리
검사 항목별 OK/NG 선택 및 측정값 입력 기능 구현
셋업 체크리스트 작성 결과 저장 기능 구현

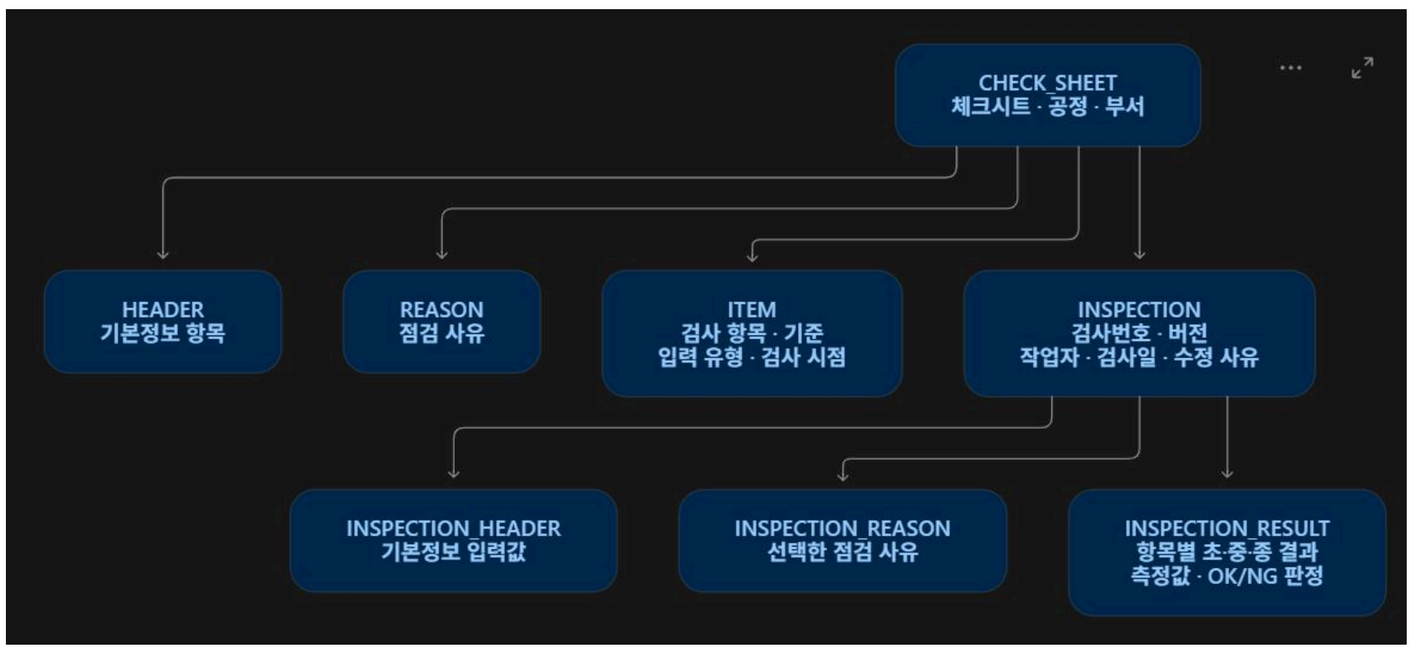
프로젝트 회고

- **공정별 양식의 공통점과 차이점 이해**
  수기 체크리스트를 전산화하면서 공정마다 기본정보, 검사 항목, 판정 기준이 다르다는 점을 확인했으며 이를 통해 화면 개발에 앞서 현장 양식과 업무 규칙을 정확히 파악하는 과정의 중요성을 배움.
- **검사 특성을 고려한 입력 방식 설계**
  같은 체크리스트 안에서도 검사 목적과 시점에 맞는 입력 방식이 필요하다는 점을 배움
- **현장 작업 흐름을 고려한 화면 구현**
  체크시트 선택부터 기본정보 입력, 검사 결과 작성, 저장까지 이어지는 흐름을 구현했습니다. 기존 양식을 화면으로 옮기는 과정에서 작업자가 검사 기준을 확인하고 결과를 입력하는 순서를 함께 고려하는 경험을 쌓음.

썬텍(베트남 공장 신규 mes 구축)

Project - 2

테이블 구조





SHEET 테이블은 PM은 준 초안을 토대로 작성함.

UTB_CHECK_SHEET 테이블을 기준으로 여러개의 헤더, 점검사유, 검사항목을 1:N 형식으로 만듬.

체크시트 작성 결과를 저장하는 UTB_CHECK_SHEET_INSPECTION 테이블을 직접 설계했습니다. 검사번호와 버전을 기준으로 기본정보, 점검 사유, 검사 결과를 연결하여 작성 결과와 수정 이력을 관리했습니다. 사용자는 소속 생산팀과 설비를 기준으로 검사 실적 목록을 조회하고, 원하는 실적을 선택하여 상세 결과를 확인할 수 있도록 구현했습니다.

프로시져

검사 시점에 따른 저장 대상 검증 --USP_PDA_CHECK_SHEET_INSPECTION_I

1. 메인 로직

```sql
SELECT M.ITEM_ID, G.Stage, M.INPUT_TYPE
FROM dbo.UTB_CHECK_SHEET_ITEM M
CROSS JOIN (VALUES (0), (1), (2)) G(Stage)
WHERE M.CHECK_SHEET_ID = @Sheet
  AND M.VALID = 1
  AND (M.INSPECTION_TIMING = 0 OR G.Stage = 0);
```

검사 시점 설정을 기준, CROSS JOIN을 토대로 VALUE 0은 초품, 1은 중품, 2는 종품으로 검사 대상을 구성함.

옆의 데이터는 @EXPECTED라는 임시 테이블에 넣어져서 @v실제 값이랑 비교해서 필수 결과 누락, 대상에 없는 항목, 단계, 입력 유형에 맞지 않는 판정값을 검증

2. 예외 사항-데이터 형식이나 결과값이 있는지

```sql
IF EXISTS (
    SELECT 1
    FROM @Expected E
    WHERE NOT EXISTS (
        SELECT 1 FROM @V V
        WHERE V.ID = E.ID
          AND TRY_CONVERT(INT, V.Stage) = E.Stage
    )
)
OR EXISTS (
    SELECT 1
    FROM @V V
    WHERE NOT EXISTS (
        SELECT 1 FROM @Expected E
        WHERE E.ID = V.ID
          AND E.Stage = TRY_CONVERT(INT, V.Stage)
    )
)
    THROW 50001, N'검사 대상과 입력 결과가 일치하지 않습니다.', 1;
```

검사 대상과 입력 결과를 양방향으로 비교하여 필수 항목,단계의 누락뿐 아니라, 대상에 없는 항목이나 불필요한 단계가 전달된 경우도 저장을 차단힘.

예로 들면, 직접 입력 값인데 OK/NG값이 들어 갔는지, 중품이나 종품 결과가 있는지, 해당 단계에서 빠진 항목이 없는지 확인함

썬텍(베트남 공장 신규 mes 구축)

Project - 2

3. 입력 데이터와 기준정보의 예외사항 검증(세부적인 값 확인)

구분	주요 검증 내용
필수값·길이	
저장자 및 체크시트·공정·부서 ID 누락, 입력 길이 초과	전달 형식
마지막 구분자 누락, 기본정보·검사 결과의 필드 개수 오류	중복 입력
기본정보 ID 또는 동일 검사 항목·단계 중복	업무 연결
체크시트와 공장·생산팀·공정 연결 및 관련 정보의 사용 여부	기본정보·점검 사유
기본정보 구성 불일치, 유효하지 않거나 누락된 점검 사유	검사 설정
허용되지 않은 입력 유형·검사 시점·검사 단계	판정값
직접 입력 항목의 불필요한 판정 코드, OK/NG 값과 코드 불일치	

4. 검사번호 생성 및 일괄 저장--중복 방지

```sql
SET @Prefix = 'SI' + CONVERT(CHAR(8), @Now, 112);
SET @Resource = N'PDA_CHECK_SHEET_NUMBER_' + @Prefix;

EXEC @LockResult = sys.sp_getapplock
    @Resource = @Resource,
    @LockMode = 'Exclusive',
    @LockOwner = 'Transaction',
    @LockTimeout = 10000;

IF @LockResult < 0
    THROW 50001, N'검사번호 채번 잠금을 획득하지 못했습니다.', 1;
```

```sql
SELECT @Next =
    ISNULL(MAX(TRY_CONVERT(INT, RIGHT(INSPECTION_NO, 6))), 0) + 1
FROM dbo.UTB_CHECK_SHEET_INSPECTION WITH (UPDLOCK, HOLDLOCK)
WHERE INSPECTION_NO LIKE @Prefix + '%'
  AND LEN(INSPECTION_NO) = 16;

IF @Next > 999999
    THROW 50001, N'일일 검사번호 범위를 초과했습니다.', 1;

SET @No =
    @Prefix + RIGHT('000000' + CONVERT(VARCHAR(6), @Next), 6);
```

여러 단말이 동시에 마지막 번호를 조회하면 같은 다음 번호를 생성함.
이를 방지하기 위해 날짜별 잠금을 적용하여, 한 요청이 번호 생성과 저장을 마칠 때까지 다른 요청은 기다리도록 처리함.
10초 내 잠금을 획득하지 못하면 오류를 반환함.

sys.sp_getapplock은 함수가 아니라 SQL Server가 제공하는 시스템 프로시저임.

해당 날짜의 마지막 일련번호에 1을 더하여 SI + 날짜 + 6자리 일련번호 형식으로 검사번호를 생성하며
잠금 획득에 실패하거나 일일 번호 범위를 초과하면 저장을 차단함.